PROJECT BASED EXPERIMENTAL LEARNING

COURSE- DATABASE MANAGEMENT SYSTEM (NoSQL / MongoDB)

PROJECT REPORT

TOPIC- MONGODB STUDENT DATABASE OPERATIONS



SUBMITTED TO- MR. Harendra Singh Rajpoot

Submitted by- Vipul Pandey

BCADS27

Roll no. - 1250258507

3rd semester

Babu Banarasi Das University

Lucknow, U.P

ACKNOWLEDGMENT

I would like to express my special thanks of gratitude to my mentor, who gave me the golden opportunity to do this wonderful project on MongoDB Student Database Operations. My mentor helped me a lot in completing this project within the given time frame, and I learned many new things about NoSQL databases from this experience.

Secondly, I want to show my gratitude to the institution that provided this project-based learning experience; it helped me to achieve my goals more efficiently.

Abstract

This report presents a MongoDB-based Student Database Management project that demonstrates the core Create, Read, Update and Delete (CRUD) operations of a NoSQL document database. A collection named "HARSH" inside the database "PANDEY" is used to store student records containing roll number, name, age, department and marks. The project implements single and bulk insertion, conditional updates, single and bulk deletion, and a wide range of query operations using MongoDB comparison and logical operators such as $gt, $lt, $eq, $gte, $ne, $and, $or and $not. The implementation demonstrates how MongoDB can be used to efficiently manage, filter and retrieve structured student data.

Introduction

MongoDB is a popular NoSQL, document-oriented database that stores data as flexible, JSON-like documents rather than in rigid rows and columns. This makes it well suited for applications where the data structure may evolve over time, such as academic record-keeping systems. This project focuses on building and querying a collection that stores academic details of 51 students (roll numbers 101–151), and demonstrates the standard database operations — insertion, updation, deletion and querying — that form the backbone of any real-world database-driven application.

Methodology

The project was carried out step by step inside the MongoDB shell (mongosh), with every operation executed against a single collection named "HARSH" in the database "PANDEY". The complete methodology, along with the exact code executed and the output produced by MongoDB at each step, is given below.

1. db.HARSH.insertOne({ rollNo: 101, name: "VIPUL", age: 22, department: "CSE", marks: 87 })

```
> use("PANDEY")
< switched to db PANDEY
> db["HARSH"].find()
<
>   db.HARSH.insertOne({ rollNo: 101, name: "VIPUL", age: 22, department: "CSE", marks: 87 })
< {
    acknowledged: true,
    insertedId: ObjectId('6ac5c0fb0b19936cd136351a')
  }
```

2. db.HARSH.insertMany([{ rollNo: 102, name: "Rahul", age: 20, department: "IT", marks: 78 }, … , { rollNo: 151, name: "Simran", age: 20, department: "ECE", marks: 83 }]) – 50 documents (roll no. 102–151)

```
db.HARSH.insertMany([ { rollNo: 102, name: "Rahul", age: 20, department: "IT", marks: 78 }, { rollNo: 103, name: "Kunal", age: 22, department: "CSE
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ac5c1b50b19936cd136351b'),
    '1': ObjectId('6ac5c1b50b19936cd136351c'),
    '2': ObjectId('6ac5c1b50b19936cd136351d'),
    '3': ObjectId('6ac5c1b50b19936cd136351e'),
    '4': ObjectId('6ac5c1b50b19936cd136351f'),
    '5': ObjectId('6ac5c1b50b19936cd1363520'),
    '6': ObjectId('6ac5c1b50b19936cd1363521'),
    '7': ObjectId('6ac5c1b50b19936cd1363522'),
    '8': ObjectId('6ac5c1b50b19936cd1363523'),
    '9': ObjectId('6ac5c1b50b19936cd1363524'),
    '10': ObjectId('6ac5c1b50b19936cd1363525'),
    '11': ObjectId('6ac5c1b50b19936cd1363526'),
    '12': ObjectId('6ac5c1b50b19936cd1363527'),
    '13': ObjectId('6ac5c1b50b19936cd1363528'),
    '14': ObjectId('6ac5c1b50b19936cd1363529'),
    '15': ObjectId('6ac5c1b50b19936cd136352a'),
    '16': ObjectId('6ac5c1b50b19936cd136352b'),
    '17': ObjectId('6ac5c1b50b19936cd136352c'),
    '18': ObjectId('6ac5c1b50b19936cd136352d'),
    '19': ObjectId('6ac5c1b50b19936cd136352e'),
```

```
'20': ObjectId('6ac5c1b50b19936cd136352f'),
'21': ObjectId('6ac5c1b50b19936cd1363530'),
'22': ObjectId('6ac5c1b50b19936cd1363531'),
'23': ObjectId('6ac5c1b50b19936cd1363532'),
'24': ObjectId('6ac5c1b50b19936cd1363533'),
'25': ObjectId('6ac5c1b50b19936cd1363534'),
'26': ObjectId('6ac5c1b50b19936cd1363535'),
'27': ObjectId('6ac5c1b50b19936cd1363536'),
'28': ObjectId('6ac5c1b50b19936cd1363537'),
'29': ObjectId('6ac5c1b50b19936cd1363538'),
'30': ObjectId('6ac5c1b50b19936cd1363539'),
'31': ObjectId('6ac5c1b50b19936cd136353a'),
'32': ObjectId('6ac5c1b50b19936cd136353b'),
'33': ObjectId('6ac5c1b50b19936cd136353c'),
'34': ObjectId('6ac5c1b50b19936cd136353d'),
'35': ObjectId('6ac5c1b50b19936cd136353e'),
'36': ObjectId('6ac5c1b50b19936cd136353f'),
'37': ObjectId('6ac5c1b50b19936cd1363540'),
'38': ObjectId('6ac5c1b50b19936cd1363541'),
'39': ObjectId('6ac5c1b50b19936cd1363542'),
'40': ObjectId('6ac5c1b50b19936cd1363543'),
'41': ObjectId('6ac5c1b50b19936cd1363544'),
'42': ObjectId('6ac5c1b50b19936cd1363545'),
'43': ObjectId('6ac5c1b50b19936cd1363546'),
'44': ObjectId('6ac5c1b50b19936cd1363547'),
'45': ObjectId('6ac5c1b50b19936cd1363548'),
'46': ObjectId('6ac5c1b50b19936cd1363549'),
'47': ObjectId('6ac5c1b50b19936cd136354a'),
'48': ObjectId('6ac5c1b50b19936cd136354b'),
```

3. db.HARSH.updateOne({ rollNo: 101 }, { $set: { marks: 90 } })

```
db.HARSH.updateOne( { rollNo: 101 }, { $set: { marks: 90 } }
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

4. db.HARSH.updateMany({ department: "CSE" }, { $set: { status: "Active" } })

```
db.HARSH.updateMany( { department: "CSE" }, { $set: { status: "Active" } } )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 17,
  modifiedCount: 17,
  upsertedCount: 0
```

5. db.HARSH.deleteOne({ rollNo: 151 })

```
db.HARSH.deleteOne({ rollNo: 151 })
{
  acknowledged: true,
  deletedCount: 1
}
```

6. db.HARSH.deleteMany({ marks: { $lt: 40 } })

```
db.HARSH.deleteMany({ marks: { $lt: 40 } })
{
  acknowledged: true,
  deletedCount: 0
}
```

7. db.HARSH.find({ marks: { $gt: 80 } })

```
> db.HARSH.find({ marks: { $gt: 80 } })
< {
    _id: ObjectId('6ac5c0fb0b19936cd136351a'),
    rollNo: 101,
    name: 'VIPUL',
    age: 22,
    department: 'CSE',
    marks: 90,
    status: 'Active'
  }
  {
    _id: ObjectId('6ac5c1b50b19936cd136351c'),
    rollNo: 103,
    name: 'Kunal',
    age: 22,
    department: 'CSE',
    marks: 92,
    status: 'Active'
  }
  {
    _id: ObjectId('6ac5c1b50b19936cd136351d'),
    rollNo: 104,
    name: 'Meera',
    age: 21,
    department: 'ECE',
    marks: 81
  }
```

8. db.HARSH.find({ marks: { $lt: 80 } })

```
db.HARSH.find({ marks: { $lt: 80 } })
{
  _id: ObjectId('6ac5c1b50b19936cd136351b'),
  rollNo: 102,
  name: 'Rahul',
  age: 20,
  department: 'IT',
  marks: 78
}
{
  _id: ObjectId('6ac5c1b50b19936cd136351e'),
  rollNo: 105,
  name: 'Varun',
  age: 23,
  department: 'ME',
  marks: 74
}
{
  _id: ObjectId('6ac5c1b50b19936cd1363520'),
  rollNo: 107,
  name: 'Rakesh',
  age: 21,
  department: 'IT',
  marks: 69
}
```

9. db.HARSH.find({ marks: { $eq: 85 } })

```
db.HARSH.find({ marks: { $eq: 85 } })
{
  _id: ObjectId('6ac5c1b50b19936cd136353b'),
  rollNo: 134,
  name: 'Shweta',
  age: 20,
  department: 'IT',
  marks: 85
}
```

10. db.HARSH.find({ marks: { $gte: 80 } })

```
> db.HARSH.find({ marks: { $gte: 80 } })
< {
    _id: ObjectId('6ac5c0fb0b19936cd136351a'),
    rollNo: 101,
    name: 'VIPUL',
    age: 22,
    department: 'CSE',
    marks: 90,
    status: 'Active'
  }
  {
    _id: ObjectId('6ac5c1b50b19936cd136351c'),
    rollNo: 103,
    name: 'Kunal',
    age: 22,
    department: 'CSE',
    marks: 92,
    status: 'Active'
  }
  {
    _id: ObjectId('6ac5c1b50b19936cd136351d'),
    rollNo: 104,
    name: 'Meera',
    age: 21,
    department: 'ECE',
    marks: 81
  }
```

11. db.HARSH.find({ marks: { $ne: 85 } })

```
db.HARSH.find({ marks: { $ne: 85 } })
{
  _id: ObjectId('6ac5c0fb0b19936cd136351a'),
  rollNo: 101,
  name: 'VIPUL',
  age: 22,
  department: 'CSE',
  marks: 90,
  status: 'Active'
}
{
  _id: ObjectId('6ac5c1b50b19936cd136351b'),
  rollNo: 102,
  name: 'Rahul',
  age: 20,
  department: 'IT',
  marks: 78
}
{
  _id: ObjectId('6ac5c1b50b19936cd136351c'),
  rollNo: 103,
  name: 'Kunal',
  age: 22,
  department: 'CSE',
  marks: 92,
  status: 'Active'
}
```

12. db.HARSH.find({ $and: [{ marks: { $gt: 70 } }, { age: { $lt: 23 } }] })

```
db.HARSH.find({ $and: [ { marks: { $gt: 70 } }, { age: { $lt: 23 } } ] })
{
  _id: ObjectId('6ac5c0fb0b19936cd136351a'),
  rollNo: 101,
  name: 'VIPUL',
  age: 22,
  department: 'CSE',
  marks: 90,
  status: 'Active'
}
{
  _id: ObjectId('6ac5c1b50b19936cd136351b'),
  rollNo: 102,
  name: 'Rahul',
  age: 20,
  department: 'IT',
  marks: 78
}
{
  _id: ObjectId('6ac5c1b50b19936cd136351c'),
  rollNo: 103,
  name: 'Kunal',
  age: 22,
  department: 'CSE',
  marks: 92,
  status: 'Active'
}
```

13. db.HARSH.find({ $or: [{ department: "CSE" }, { department: "ECE" }] })

```
db.HARSH.find({ $or: [ { department: "CSE" }, { department: "ECE" } ] })
{
  _id: ObjectId('6ac5c0fb0b19936cd136351a'),
  rollNo: 101,
  name: 'VIPUL',
  age: 22,
  department: 'CSE',
  marks: 90,
  status: 'Active'
}
{
  _id: ObjectId('6ac5c1b50b19936cd136351c'),
  rollNo: 103,
  name: 'Kunal',
  age: 22,
  department: 'CSE',
  marks: 92,
  status: 'Active'
}
{
  _id: ObjectId('6ac5c1b50b19936cd136351d'),
  rollNo: 104,
  name: 'Meera',
  age: 21,
  department: 'ECE',
  marks: 81
}
```

14. `db.HARSH.find({ marks: { $not: { $gt: 80 } } })`

```
db.HARSH.find({ marks: { $not: { $gt: 80 } } })
{
  _id: ObjectId('6ac5c1b50b19936cd136351b'),
  rollNo: 102,
  name: 'Rahul',
  age: 20,
  department: 'IT',
  marks: 78
}
{
  _id: ObjectId('6ac5c1b50b19936cd136351e'),
  rollNo: 105,
  name: 'Varun',
  age: 23,
  department: 'ME',
  marks: 74
}
{
  _id: ObjectId('6ac5c1b50b19936cd1363520'),
  rollNo: 107,
  name: 'Rakesh',
  age: 21,
  department: 'IT',
  marks: 69
}
```

8. MongoDB Operator Summary

Operator	Meaning	Example
$gt	Greater than	marks: { $gt: 80 }
$lt	Less than	marks: { $lt: 80 }
$eq	Equal to	marks: { $eq: 85 }
$gte	Greater than or equal to	marks: { $gte: 80 }
$ne	Not equal to	marks: { $ne: 85 }
$and	All conditions must be true	marks > 70 AND age < 23
$or	At least one condition must be true	CSE OR ECE
$not	Negates a condition	NOT (marks > 80)

Total dataset: 51 students inserted (Roll No. 101–151); 50 remain after deleteOne({ rollNo: 151 }).

Implementation Details

- The project is implemented using MongoDB and executed through the MongoDB Shell (mongosh) / Compass.
- A single collection named "HARSH" in the database "PANDEY" is used, with fields: rollNo, name, age, department and marks.
- insertOne() and insertMany() are used to populate the collection with 51 student records.
- updateOne() and updateMany() with the $set operator are used to modify existing fields and add new ones (e.g. status).
- deleteOne() and deleteMany() are used to remove individual and bulk records based on a filter condition.
- find() is combined with comparison operators ($gt, $lt, $eq, $gte, $ne) and logical operators ($and, $or, $not) to retrieve specific subsets of student data.
- Each operation's acknowledgement object (acknowledged, matchedCount, modifiedCount, deletedCount) is used to verify that it executed as intended.

Results

All CRUD operations executed successfully on the HARSH collection. The insertOne and insertMany operations populated the collection with 51 student records. updateOne changed the marks of roll no. 101 (VIPUL) from 87 to 90 (matchedCount 1, modifiedCount 1), while updateMany matched and modified all 17 CSE-department documents to add an "Active" status. deleteOne removed the record of roll no. 151 (deletedCount 1), leaving 50 documents, and deleteMany found no records with marks below 40, correctly reporting a deletedCount of 0.

On the remaining 50 documents, the query operations returned: marks > 80 — 27 students; marks < 80 — 22; marks = 85 — 1 (Shweta, IT); marks >= 80 — 28; marks != 85 — 49; marks > 70 AND age < 23 — 37; department CSE OR ECE — 29; NOT (marks > 80) — 23. These counts are consistent with the dataset, showing that MongoDB's query engine returns accurate, condition-based results.

Observations from the data: the average mark is about 81.4; CSE has the highest department average (≈85.5) and the topper is Mansi (CSE, 97), while ME has the lowest average (≈71.3) and the lowest mark is 65 (Sachin, ME).

Conclusion

This project highlights the practical use of MongoDB as a NoSQL database for managing structured academic data. By implementing insertion, update, deletion and a wide variety of query operations on a student collection, the project demonstrates a solid, hands-on understanding of MongoDB's Create, Read, Update and Delete (CRUD) capabilities as well as its comparison and logical query operators. The approach shows how a document database can be used to store and efficiently retrieve real-world tabular data such as student records.

Future Scope

- Use the MongoDB Aggregation Framework to compute department-wise averages, toppers and rankings.
- Add schema validation rules to enforce data types and required fields at the database level.
- Create indexes on frequently queried fields (e.g. marks, department) to improve performance.
- Build a front-end / web dashboard (e.g. using Node.js and Express) to interact with the database visually.
- Extend the collection with additional entities such as courses, attendance and results, and model relationships between them.

References

- MongoDB Official Documentation: https://www.mongodb.com/docs/
- MongoDB CRUD Operations: https://www.mongodb.com/docs/manual/crud/
- MongoDB Query Operators: https://www.mongodb.com/docs/manual/reference/operator/query/
- mongosh (MongoDB Shell) Documentation: https://www.mongodb.com/docs/mongodb-shell/